

# auto dsp task config guide

## 1.背景

Auto adsp 中会存在多个 task (bus out)，且每个 task 根据需求及实际场景不同，需要配置不同的算法处理参数，例如 channel in，channel out 及是否 bypass 算法等。同时，一般需要将各路 bus 的 channel 映射到最终输出的 TDM 格式中对应的 channel 位置，即需要配置 channel map 信息，故提供 audio\_dsp\_config.xml 配置修改 auto dsp task 的各项属性及 mix path。一般的 adsp 中的 task 架构如图：

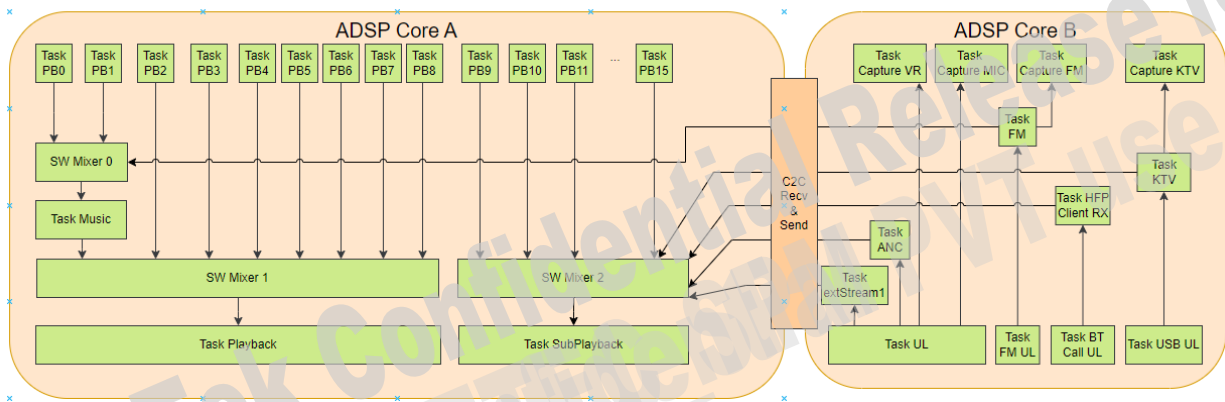


Figure 1 adsp architecture

## 2.Xml 配置说明

- 路径

Xml 在设备上的 path 如下： vendor/etc/audio\_dsp\_config.xml

在 code 中 path 如下： device/<project\_name>/ audio\_dsp\_config.xml

### 2.1 mixer\_target & mixer\_source

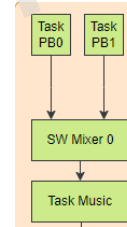
xml 中将每个 dsp task 根据其在 dsp 中的 mixer 的位置分为了 sw\_mixer\_target 和 sw\_mixer\_source。对于 Task Music，其即是 sw\_mixer\_target，也是 sw\_mixer\_source。

- sw\_mixer\_target  
对应于 sw\_mixer 的输出端，如 Task Music, Task Playback, Task SubPlayback
- sw\_mixer\_source  
对应于 sw\_mixer 的输入端，如 Task PB0 (name="sw\_mixer\_playback0")

task 作为 target 或者 source 是不可配置的，其角色是预定好的，但是可以通过 xml 配置 sw\_mixer\_source mix 到期望的 target 中。对应的修改方法是在 xml 中，将对应 name 的 sw\_mixer\_source 节点写到对应的 sw\_mixer\_target 的节点下，如下例子表示 task PB0 (sw\_mixer\_playback0) 将会 mix 到 task music(sw\_mixer\_music)中。

Ex1:

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16" aurosys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurosys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```



目前可以配置的 sw\_mixer\_source 有: sw\_mixer\_playback0 ~ sw\_mixer\_playback15, sw\_mixer\_FM\_ADSP, sw\_mixer\_hfp\_client\_rx, sw\_mixer\_anc, sw\_mixer\_extstream1, sw\_mixer\_extstream2, sw\_mixer\_ktv

## 2.2 task attribute

在每个 dsp task，由于场景需要，会配置不同的算法输入输出 channel 及 channel map 信息。在 xml 中，可以为每个 task 分别配置下列属性：

- for sw\_mixer\_target
  - ch\_in: Task 的算法输入 channel
  - ch\_out: Task 的算法输出 channel
  - aurosys\_on: 是否开启 Task 算法，仅在 ch\_in = ch\_out 时可配置成 0

例如，在下面例子中，task music 的算法输入 channel 是 2，算法处理后的输出 channel 是 16，算法非 bypass。

```
<sw_mixer_target name="sw_mixer_music" ch_in="2" ch_out="16" aurosys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurosys_on="0"
      ch_map="0x5555,0xaaaa"/>
  </sw_mixer_source>
</sw_mixer_target>
```

- for sw\_mixer\_source，每个 profile 有如下属性

- ch\_in: Task 的算法输入 channel
- ch\_out: Task 的算法输出 channel
- aurosys\_on: 是否开启 Task 算法，仅对 pb0~pb15 在 ch\_in = ch\_out 时配置成 0 可生效
- ch\_map: source 在向 sw\_mixer\_target 映射时候的 channel map 信息，是一个数组，数组中的 index 表示 source 的 channel index，对应的值是一个 16bit 的 value，每一个 bit 表示是否向 target 对应 channel 映射。如 ch\_map="0x5555,0xaaaa"，ch\_map[0] = 0x5555，表示 source 的 channel 1 映射数值为 0x5555，对应 2 进制为 0101010101010101，则向 target 的 1,3,5,7,9,11,13,15 channel 映射。ch\_map[1] = 0xaaaa，对应二进制位 1010101010101010，source 的 channel 2 向 target 的 2,4,6,8,10,12,14,16 映射。

aurisys\_on为0时表示关闭，仅pb0~pb15  
在 ch\_in = ch\_out 时可配置成0

对 source task，一般其输入 channel 是由数据原始端给出的，如 audioflinger 送下来，可能是任意 channel 数量，所以在 source task 中，xml 可配置支持不同的 ch\_in 的多种 profile，可以根据不同的 ch\_in 选择不同的 ch\_out, ch\_map 等信息。例如，如下 task pb14 具有两个 profile，当进入 adsp 给到 task pb14 的 channel 是 2ch，则使用第一个 profile，如果是 12ch，则使用第二个。

```
<sw_mixer_source name="sw_mixer_playback14">
  <profile ch_in="2" ch_out="2" aurisys_on="0"
    ch_map="0x5555,0xaaaa"/>
  <profile ch_in="12" ch_out="12" aurisys_on="0"
    ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800,0x1000,0x2000,0x4000,0x8000"/>
</sw_mixer_source>
```

对于 music task，由于其即是 source，也是 target，则会取其 target 属性中的 ch\_in 属性，在 source 中找到对应的 profile 作为其配置。

### 3. 配置示例

这里给出一个 task pb0 配置示例如下：

```
<sw_mixer_target name="sw_mixer_music" ch_in="8" ch_out="12" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_playback0">
    <profile ch_in="2" ch_out="2" aurisys_on="0"
      ch_map="0x4,0x8"/>
    <profile ch_in="4" ch_out="4" aurisys_on="0"
      ch_map="0x1,0x2,0x4,0x8"/>
  </sw_mixer_source>
</sw_mixer_target>

<sw_mixer_target name="sw_mixer_playback" ch_in="16" ch_out="16" aurisys_on="1">
  <sw_mixer_source name="sw_mixer_music">
    <profile ch_in="8" ch_out="12" aurisys_on="1"
      ch_map="0x1,0x2,0x4,0x8,0x10,0x20,0x40,0x80,0x100,0x200,0x400,0x800"/>
  </sw_mixer_source>
</sw_mixer_target>
```

对应的数据通路流程如下图：

